

Week 3: The Analog World

Mastering PWM & Potentiometers to Bridge the Gap Between Digital Systems
and the Physical Universe.

Show
Animation

Signals: Analog vs. Digital

The Analog Universe

Nature operates in analog. Sound waves, light intensity, and temperature are continuous signals. They can hold **infinite possible values** within a given range, smoothly transitioning from one state to another.



The Digital Reality

Microcontrollers operate digitally. They understand the world in discrete, absolute binary states: **HIGH (1)** or **LOW (0)**. To interface with the real world, we must translate between these two distinct paradigms.



Demystifying **Arduino** I/O

Understanding ADC, PWM, and Hardware Timers in the ATmega328

Microcontroller

Read vs. Write Limits

analogWrite()

This standard output function is hardware-restricted to values ranging precisely from **0 to 255**.

It utilizes an 8-bit timer to generate Pulse Width Modulation (PWM) signals, rapidly pulsing a digital pin to simulate an analog voltage.

analogRead()

This input function returns much higher-resolution values ranging from **0 to 1023**.

It relies on a 10-bit Analog-to-Digital Converter (ADC) embedded inside the chip to read incoming voltage with fine precision.

Pulse Width Modulation

Faking Analog Voltage with Digital
Speed

How PWM Works

The "Rapid Switch"

An Arduino digital pin can only output 5V (ON) or 0V (OFF). PWM allows us to fake a middle ground by turning the signal on and off incredibly fast.

- **Duty Cycle:** The precise percentage of time the digital signal is ON within one cycle.
- **0% Duty Cycle:** Always off (0V).
- **50% Duty Cycle:** On half the time (Averages out to 2.5V).
- **100% Duty Cycle:** Always on (5V).

50% duty cycle



75% duty cycle



25% duty cycle



Show
Animation

Implementing `analogWrite()`

The Hardware: PWM Pins

You cannot use PWM on every single digital pin. Look closely at your Arduino board for the pins marked with a tilde symbol (~).

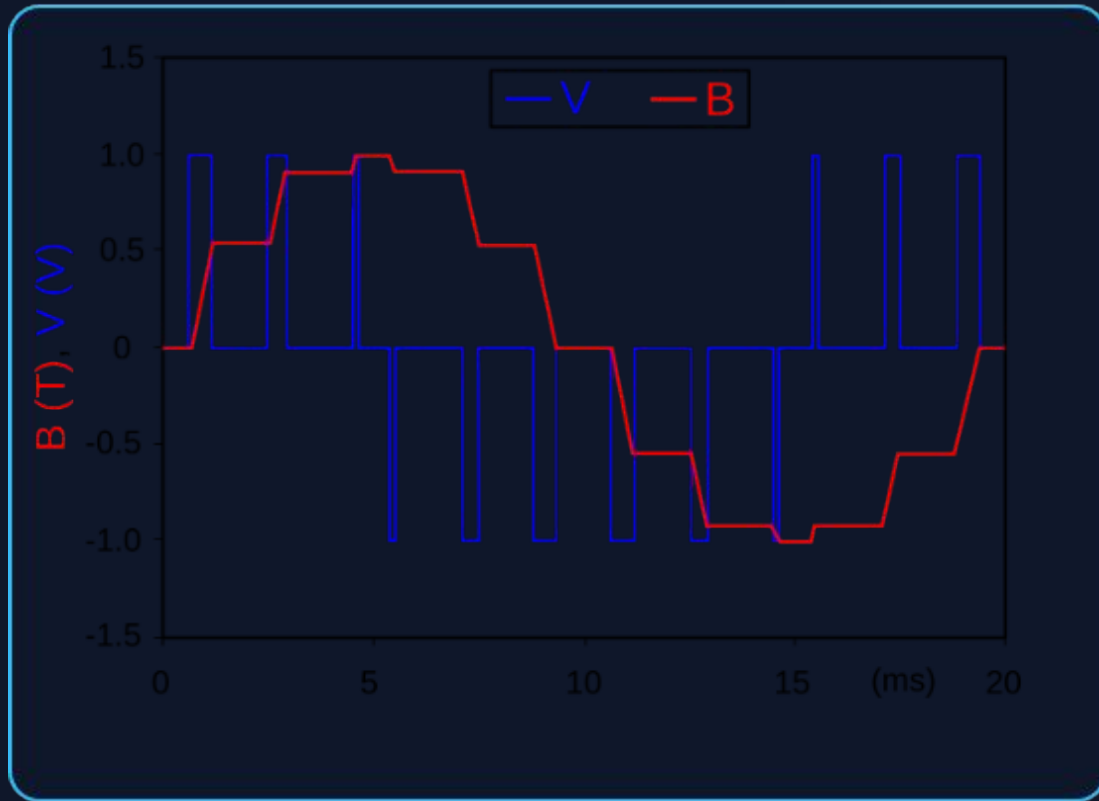
- **Arduino Uno PWM Pins:** Digital pins 3, 5, 6, 9, 10, and 11.
- Connecting an LED or a motor driver to these specific pins allows for variable control rather than just simple ON/OFF states.

The Software: 8-Bit Range

The `analogWrite(pin, value)` function uses an internal 8-bit timer. In binary, 8 bits means you have 256 distinct levels.

- **Value 0:** Fully off (0% Duty Cycle / 0V).
- **Value 127:** ~50% Duty Cycle (~2.5V).
- **Value 255:** Fully on (100% Duty Cycle / 5V).

The 8-bit PWM



`analogWrite()` does not output true analog voltage. It relies on Pulse Width Modulation (PWM) to rapidly switch pins ON and OFF.

The standard hardware timers controlling this operate on an **8-bit scale**.

$$2^8 = 256$$

A value of 0 is always OFF (0V), and 255 is always ON (5V). A middle value like 127 simulates roughly 2.5V output.

Show
Animation

Analog to Digital Converter

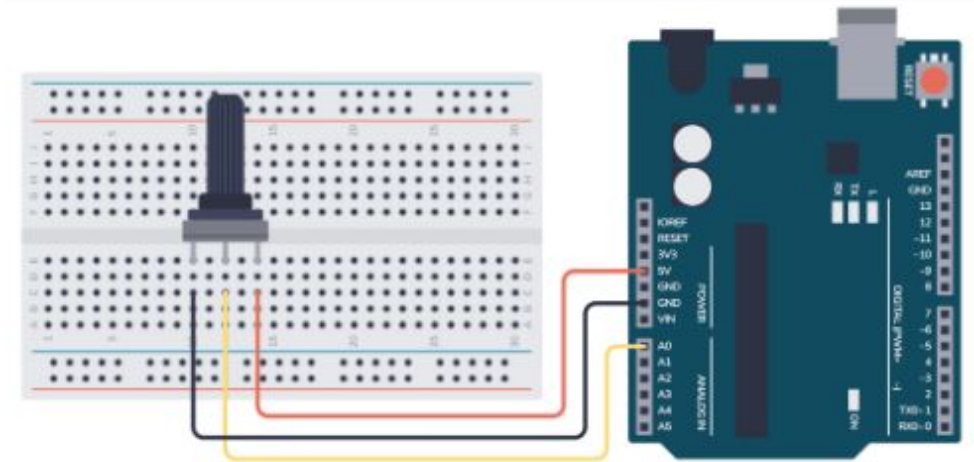
Reading and Measuring the Real
World

How `analogRead()` Works

The Digital Ruler

If PWM is about "sending" a fake analog voltage, `AnalogRead` is about "listening" to a real continuous voltage from the outside world.

- **The ADC Tool:** The Arduino has an Analog-to-Digital Converter (ADC) that acts like a ruler, measuring voltages between 0V and 5V.
- **Analog Pins:** You must plug inputs into the dedicated headers labeled **A0 through A5**.
- **Classic Use Case:** Twisting a potentiometer changes the voltage sent to the pin. The Arduino reads that change.



The ADC Resolution Math

1024

DISTINCT STEPS

10-Bit Conversion

The Arduino Uno has a 10-bit ADC. This breaks the smooth 0-5V analog range into exactly 1,024 digital steps (values ranging from 0 to 1023). To find the true physical voltage, we use this math formula:

$$\text{Voltage} = \frac{\text{Reading} \times 5\text{V}}{1023}$$

Hardware: The ADC

Analog-to-Digital Converter

Microcontrollers cannot natively understand an analog voltage like 3.14V. They require hardware to bridge the gap.

- ▶ The ADC acts as a **translator** between the physical world and the digital brain.
- ▶ It reads a continuous voltage on a specific pin (like A0) and converts it into a discrete numeric value.
- ▶ Without an ADC, sensors measuring light, temperature, or position would be useless to the Arduino.



Understanding 10-Bit Resolution

1024

Discrete Steps (0-1023)

From Voltage to Data

The standard Arduino ADC operates at a 10-bit resolution. This means it slices the 0V to 5V input range into exactly 210 (or 1024) tiny, distinct voltage steps.

- ▶ **0V Input:** Digitally read as **0**
- ▶ **2.5V Input:** Digitally read as **512**
- ▶ **5V Input:** Digitally read as **1023**

Show
Animation

Key Differences: Read vs. Write

Feature / Function	<code>analogRead()</code>	<code>analogWrite()</code> (PWM)
Data Direction	Input (Arduino "listens")	Output (Arduino "talks")
Compatible Pins	Only headers A0 – A5	Digital pins with ~ (3, 5, 6, 9, 10, 11)
Value Range	0 to 1023 (10-bit ADC)	0 to 255 (8-bit Timer)
Physical Reality	Measures actual, continuous real-world voltage	Fakes an analog voltage using high-speed pulsing

ATmega328 Hardware Timers



Timer 0

An 8-bit timer primarily responsible for keeping time and controlling PWM output on digital pins 5 and 6.



Timer 2

A secondary 8-bit timer, dedicated to controlling the PWM signals on digital pins 3 and 11.



Timer 1

A powerful 16-bit timer that controls PWM on pins 9 and 10, capable of generating far greater precision signals.

Why Restrict to 8-bit?

For consistency and simplicity.

— The Arduino Core Design Philosophy

The creators forced all standard PWM pins to operate on an 8-bit scale so users wouldn't have to memorize which specific pins use 8-bit timers versus 16-bit timers.

Unlocking 16-bit PWM

Even though Timer 1 is physically a 16-bit timer, the Arduino software artificially restricts `analogWrite()` to an 8-bit ceiling.

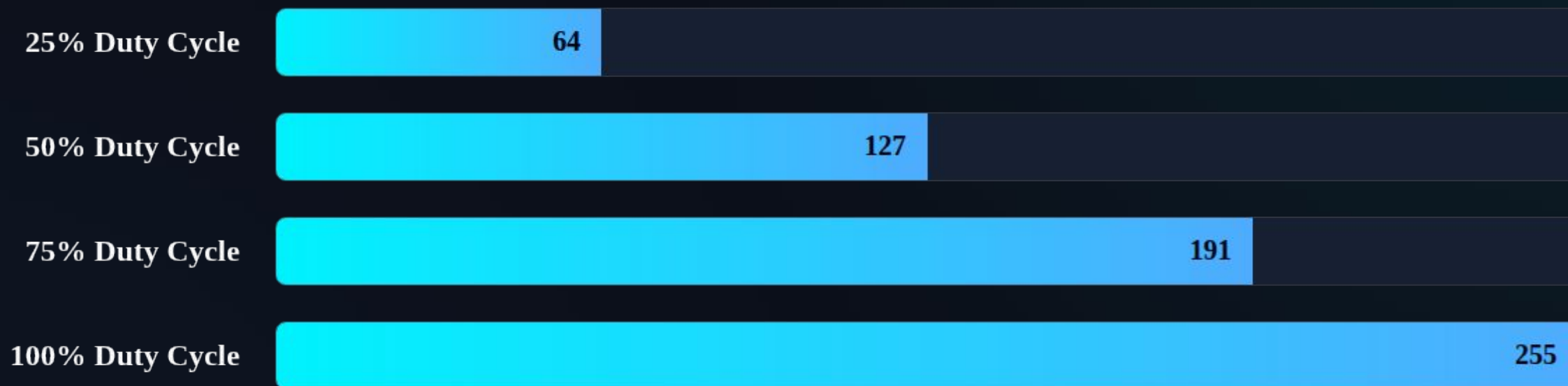
However, by writing custom code to directly access and manipulate the ATmega328's internal memory registers, you can easily bypass this safety limit.

Doing so unlocks the full **0–65535** range, which is critical for smooth servo motor control or high-fidelity custom audio applications.

Start
Arduino
Programming in
C++



PWM in Action: Duty Cycles



The `analogWrite()` function accepts values from **0 to 255**. This 8-bit resolution dictates the percentage of time the signal remains HIGH, effectively altering the perceived average voltage from 0V to 5V.

Meet the Potentiometer



The Classic Analog Input

A potentiometer is essentially a manually adjustable variable resistor with three pins. It acts as a voltage divider.

- ▶ **Outer Pins:** Connected across the power supply (5V and GND).
- ▶ **Middle Wiper:** Outputs a varying voltage depending on the physical position of the knob.
- ▶ **Result:** A smooth, continuous analog signal ready to be read by the Arduino's ADC.

Lab 1: Reading Potentiometer



1. Connect

Wire the outer pins of the potentiometer to 5V and GND. Connect the middle wiper pin directly to analog pin **A0**.



2. Code

Use the **analogRead(A0)** function in your sketch. This commands the ADC to measure the voltage and return a value (0-1023).



3. Monitor

Initialize Serial communication at 9600 baud and use **Serial.println()** to view the real-time sensor data.

Lab 2: **Fading** an LED

Dynamic Brightness Control

Now we will combine our inputs and outputs. We'll use the analog reading from the potentiometer to dynamically adjust the brightness of an LED via PWM.

- ▶ **Input:** Read potentiometer on Analog Pin A0.
- ▶ **Output:** Send PWM signal to a digital pin featuring the "~" symbol (e.g., Pin 9).
- ▶ **Result:** Twisting the knob smoothly fades the LED from off to maximum brightness.



Questions?

Ready to start prototyping your next circuit?